Abstract Data Types in Python

Y13 Computer Science — Stacks, Queues, Linked Lists and Binary Trees

Open in Colab Code and PDF: github.com/mrpiay/Y13_ADTs

This tutorial uses four small Python programs — `stack.py`, `queue.py`, `linked_list.py` and `binary_tree.py` — to show how the classic **Abstract Data Types (ADTs)** are built from nothing more than **arrays** and **integer pointers**.

By the end you should be able to:

- explain what an ADT is and why we separate *what* it does from *how* it is built;
- implement a **stack**, a **circular queue**, a **linked list** and a **binary search tree** using arrays;
- explain how a **free list** (called the *heap* in this code) recycles unused nodes;
- **trace** the contents of the arrays and pointers after a sequence of operations — a very common exam question;
- write new operations (search, count, reverse, traversals…) on top of an existing ADT;
- get a first feel for **efficiency**: which operations are fast (O(1)) and which get slower as the data grows (O(n)).

How to use it

1. Run the cells **in order** (in Colab: *Runtime* → *Run all*, or `Shift + Enter` one cell at a time).
2. Each section contains 🖍️ **exercises**: write your code in the empty cell, then run the 🧪 test cell below it.
3. Solutions are hidden in collapsed ✅ cells — try properly before you peek!

Contents

1. What is an Abstract Data Type?
2. The Stack (LIFO)
3. The Circular Queue (FIFO)
4. The Linked List
5. The Binary Search Tree
6. Exam-style tracing questions
7. Summary

Appendix A. Solutions to the exercises

Appendix B. Answers to the tracing questions

1. What is an Abstract Data Type?

An **Abstract Data Type (ADT)** is a collection of data together with a set of **operations** on that data. It is defined by **what it does**, not by **how it is built**.

	The ADT (abstract view)	**The implementation (concrete view)**
Question it answers	*What* can I do with it?	*How* is it stored and coded?
Stack example	`push`, `pop`, `peek`, `isEmpty`, `isFull`	an array `sData` and an integer `sTop`
Who cares?	the programmer **using** the stack	the programmer **writing** the stack

Because the user only depends on the operations, the implementation can be changed (array → linked nodes, Python list → database…) **without changing any code that uses it**. This is **abstraction** and **encapsulation**.

Why build ADTs with arrays?

Python already has lists, `collections.deque` and so on — but in the exam you are expected to build these structures from a **fixed-size array** and **integer pointers**, exactly as you would in a language without dynamic structures. That is precisely what our four files do.

Conventions used in all four files

Convention	**Meaning**
Prefixes `s`, `q`, `l`, `b`	stack, queue, linked list, binary tree — so names never clash
`xInit()`	creates the arrays and resets the pointers. **Always call it first.**
`xEmpty()` / `xFull()`	Boolean checks used before removing / adding
A **pointer**	just an **index** into an array (an integer)
`-1`	the **null pointer**: "points nowhere" (empty, end of list, no child)
`'-'`	placeholder meaning "this slot holds no data"
`global`	the functions change variables created in `xInit()`, so they must declare them `global`

> 💡 **Key idea:** In every structure below, the "boxes and arrows" you draw on paper are really just **numbers stored in arrays**. An arrow from node 3 to node 7 simply means "`links[3]` contains `7`".

How fast is it? A first look at Big-O

In each section you will see a 📝 **Efficiency** note saying how the time an operation takes **grows as the structure gets bigger**. We write this with **Big-O notation**. You will study it properly later in the course; for now, three cases are enough. Here **n** means *the number of items* in the structure.

Notation	Name	What it means	Everyday example
O(1)	constant	always the **same number of steps**, however big n is	taking the top plate off a pile
O(n)	linear	the steps grow **in proportion** to n: twice the items, twice the work	looking for a friend by walking along a queue, person by person
O(log n)	logarithmic	each step **halves** what is left, so doubling n adds just **one** more step	finding a word in a dictionary by opening it in the middle

To feel the difference, here is roughly how many steps each one needs:

n (items)	O(1)	O(log n)	O(n)
1 000	1	about 10	1 000
1 000 000	1	about 20	1 000 000

> 💡 **Rule of thumb:** look at the loops. **No loop** that depends on the number of items → **O(1)**. A loop that may visit **every item** once → **O(n)**. A loop that **halves** the remaining items each time → **O(log n)**.

2. The Stack (LIFO)

A **stack** is a **Last In, First Out** structure: the last item added is the first one removed — like a pile of plates.

Operation	Function	What it does
push	`sPush(value)`	add an item on top
pop	`sPop()`	remove **and return** the top item
peek	`sPeek()`	return the top item **without** removing it
isEmpty / isFull	`sEmpty()` / `sFull()`	check before popping / pushing

Where stacks are used: *undo* in editors, the browser *back* button, the **call stack** that stores return addresses for subroutines (and recursion), evaluating Reverse Polish expressions, checking brackets, backtracking (mazes).

Implementation with an array

Variable	Role
`sSize`	maximum number of items (40)
`sData`	the array holding the items
`sTop`	index of the **top** item; `-1` means the stack is empty

- **push:** check not full → `sTop = sTop + 1` → store the value at `sData[sTop]`
- **pop:** check not empty → read `sData[sTop]` → `sTop = sTop - 1`

Here is `stack.py`, unchanged. Run it to define the functions:

```
# Function to initialize the stack
def sInit():
    global sSize, sData, sTop
    sSize = 40  # The maximum size of the stack
    sData = []  # List to hold stack elements
    # Initialize the stack with default placeholder values
    for i in range(sSize):
        sData.append('-')
    sTop = -1  # Index of the topmost element in the stack; -1 indicates the stack is empty

# Function to check if the stack is empty
def sEmpty():
    return sTop == -1  # Returns True if the stack is empty, False otherwise

# Function to check if the stack is full
def sFull():
    return sTop == sSize - 1  # Returns True if the stack is full, False otherwise

# Function to print the current elements of the stack
def sPrint():
    if sEmpty():
        print("Stack is empty")  # Message when the stack is empty
    else:
        # Print the stack elements from top to bottom
        for i in range(sTop + 1):
            print(sData[sTop - i])  # Reverse order for proper stack representation

# Function to push a value onto the stack
def sPush(value):
    global sTop
    if sFull():
        print("Stack is full")  # Message when the stack is full
    else:
        sTop = sTop + 1  # Move the top index up by one
        sData[sTop] = value  # Add the new value at the top of the stack

# Function to pop the top value from the stack
def sPop():
    global sTop
    if sEmpty():
        print("Stack is empty")  # Message when the stack is empty
    else:
        sTopValue = sData[sTop]  # Retrieve the topmost value
        sTop = sTop - 1  # Move the top index down by one
        return sTopValue  # Return the popped value

# Function to peek at the top value of the stack without removing it
def sPeek():
    if sEmpty():
        print("Stack is empty")  # Message when the stack is empty
    else:
        return sData[sTop]  # Return the topmost value
```

To *see* what happens inside the array we add a small helper that prints the first slots, top first, and marks where `sTop` points:

```
def sShow(n=6):
    """Show the bottom n slots of the stack array, highest index first."""
    print("index | sData")
    print("------+------")
    for i in range(n - 1, -1, -1):
        marker = "  <-- sTop" if i == sTop else ""
        print(f"{i:>5} | {str(sData[i]):<5}{marker}")
    print("sTop =", sTop)
```

Trying it out

```
sInit()
for value in ["A", "B", "C", "D"]:
    sPush(value)
sShow()
```

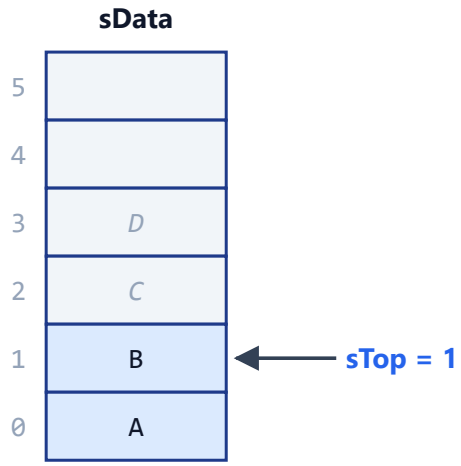
OUTPUT

```
index | sData
------+------
    5 | -
    4 | -
    3 | D      <-- sTop
    2 | C
    1 | B
    0 | A
sTop = 3
```

```
print("Popped:", sPop())
print("Popped:", sPop())
print("Peek:  ", sPeek())
sShow()
```

OUTPUT

```
Popped: D
Popped: C
Peek:   B
index | sData
------+------
    5 | -
    4 | -
    3 | D
    2 | C
    1 | B      <-- sTop
    0 | A
sTop = 1
```



The stack after two pops. C and D are still physically in the array, but they are above sTop, so they are no longer part of the stack.

> ⚠️ **Common misconception:** `sPop()` does **not** delete anything from the array — it only moves `sTop` down. `C` and `D` are still in `sData[2]` and `sData[3]`, but anything **above** `sTop` is treated as garbage and will be overwritten by the next push. Examiners love asking for the array contents after some pops!

Underflow and overflow

- **Underflow:** popping from an empty stack.
- **Overflow:** pushing onto a full stack.

```
sInit()
result = sPop()                 # underflow
print("sPop() returned:", result)

for i in range(sSize + 1):      # one push too many -> overflow
    sPush(i)
print("sTop =", sTop, "| sFull() =", sFull())
```

OUTPUT

```
Stack is empty
sPop() returned: None
Stack is full
sTop = 39 | sFull() = True
```

Notice that `sPop()` on an empty stack prints a message and then returns `None` (there is no `return` on that path). Code that uses the stack should therefore **check `sEmpty()` before popping**.

> 📝 **Efficiency:** push, pop and peek are all **O(1)** — they take the same time however many items the stack holds.

Quick check — answer in your head: 1. Why does `sTop` start at `-1` and not `0`? 2. What is the value of `sTop` when the stack is full? 3. Why does `sPush` need `global sTop` but `sEmpty` does not?

🖍️ Exercise E1 — Reverse a string with a stack

Write `reverseString(string)` that returns the string reversed, **using the stack ADT** (`sInit`, `sPush`, `sPop`, `sEmpty`). Do **not** use slicing like `string[::-1]`.

Hint: push every character, then pop until the stack is empty.

STARTER CODE

```
def reverseString(string):
    # Your code here
    pass
```

TESTS (RUN THESE IN THE NOTEBOOK)

```
check("reverseString('hello')", reverseString("hello"), "olleh")
check("reverseString('')", reverseString(""), "")
check("reverseString('Y13')", reverseString("Y13"), "31Y")
```

🖍️ Exercise E2 — Balanced brackets

A classic use of a stack. Write `isBalanced(expression)` that returns `True` if every bracket in the expression is closed in the correct order. It must handle **three** kinds of bracket: `()`, `[]` and `{}`.

- `"{[()]}"` → `True` `"([)]"` → `False` `"(()"` → `False`

Hint: push every opening bracket. When you meet a closing bracket, the stack must not be empty **and** the popped bracket must be the matching opener. At the end the stack must be empty.

STARTER CODE

```
def isBalanced(expression):
    # Your code here
    pass
```

TESTS (RUN THESE IN THE NOTEBOOK)

```
check("(())", isBalanced("(())"), True)
check("(()", isBalanced("(()"), False)
check(")(", isBalanced(")("), False)
check("{[()]}", isBalanced("{[()]}"), True)
check("([)]", isBalanced("([)]"), False)
check("a*(b+c[i])", isBalanced("a*(b+c[i])"), True)
```

3. The Circular Queue (FIFO)

A **queue** is a **First In, First Out** structure: items leave in the same order they arrived — like a queue at the canteen.

Operation	Function	What it does
enqueue	`qAdd(value)`	add an item at the **rear**
dequeue	`qRemove()`	remove the item at the **front**
isEmpty / isFull	`qEmpty()` / `qFull()`	check before removing / adding

Where queues are used: printer spools, keyboard buffers, processes waiting for the CPU, network packets, breadth-first search.

Why *circular*?

In a simple **linear** queue the front pointer keeps moving to the right as items are removed. The slots behind it are wasted, and soon the queue reports "full" even though most of the array is empty.

A **circular queue** fixes this: when a pointer reaches the end of the array it **wraps around to index 0**. The trick is the **modulo** operator `%`:

```
next = (current + 1) % qSize        # with qSize = 4:  0 → 1 → 2 → 3 → 0 → 1 …
```

Implementation with an array

Variable	Role
`qSize`	maximum number of items (only **4**, so we can see the wrap-around)
`qData`	the array holding the items
`qFront`	index of the item at the **front** (next to leave)
`qRear`	index of the item at the **rear** (last to arrive); starts at `-1`
`qCount`	how many items are in the queue

> 💡 **Why keep `qCount`?** In a circular queue, `qFront` and `qRear` alone can look the same when the queue is empty and when it is full. Counting the items makes `qEmpty()` and `qFull()` trivial and unambiguous.

Here is `queue.py`, unchanged:

```
# Function to initialize the circular queue
def qInit():
    global qSize, qData, qFront, qRear, qCount
    qSize = 4  # The maximum size of the queue
    qData = []  # List to hold the queue elements
    # Initialize the queue with default placeholder values
    for i in range(qSize):
        qData.append('-')
    qFront = 0  # Points to the front element of the queue
    qRear = -1  # Points to the last element of the queue; -1 indicates the queue is empty
    qCount = 0  # Number of elements currently in the queue

# Function to check if the queue is empty
def qEmpty():
    return qCount == 0  # Returns True if the queue is empty, False otherwise

# Function to check if the queue is full
def qFull():
    return qCount == qSize  # Returns True if the queue is full, False otherwise

# Function to print the current elements of the queue
def qPrint():
    if qEmpty():
        print("Queue is empty")  # Message when the queue is empty
    else:
        current = qFront  # Start from the front of the queue
        # Loop through the number of elements in the queue
        for i in range(qCount):
            print(qData[current])  # Print the current element
            current = (current + 1) % qSize  # Move to the next element (circularly)

# Function to add a value to the rear of the queue
def qAdd(value):
    global qRear, qCount

    if qFull():
        print("Queue is full")  # Message when the queue is full
    else:
        # Move the rear pointer to the next position (circularly)
        qRear = (qRear + 1) % qSize
        qData[qRear] = value  # Add the new value at the rear
        qCount = qCount + 1  # Increment the count of elements in the queue

# Function to remove a value from the front of the queue
# Returns the removed value, or None if the queue was empty
def qRemove():
    global qFront, qCount

    if qEmpty():
        print("Queue is empty")  # Message when the queue is empty
        return None
    else:
        qFrontValue = qData[qFront]  # Save the front value before it is overwritten
        qData[qFront] = '-'  # Reset the front position to the default placeholder
        # Move the front pointer to the next position (circularly)
        qFront = (qFront + 1) % qSize
        qCount = qCount - 1  # Decrement the count of elements in the queue
        return qFrontValue  # Return the removed value
```

```
def qShow():
    """Show the queue array together with its pointers and counter."""
    print("index :", " ".join(f"{i:>4}" for i in range(qSize)))
    print("qData :", " ".join(f"{str(x):>4}" for x in qData))
    print(f"qFront = {qFront}, qRear = {qRear}, qCount = {qCount}")
```

Trying it out — watch the wrap-around

```
qInit()
for value in [10, 20, 30]:
    qAdd(value)
qShow()
```

OUTPUT

```
index :    0    1    2    3
qData :   10   20   30    -
qFront = 0, qRear = 2, qCount = 3
```

```
print("Removed:", qRemove())
print("Removed:", qRemove())
qShow()
```

OUTPUT

```
Removed: 10
Removed: 20
index :    0    1    2    3
qData :    -    -   30    -
qFront = 2, qRear = 2, qCount = 1
```

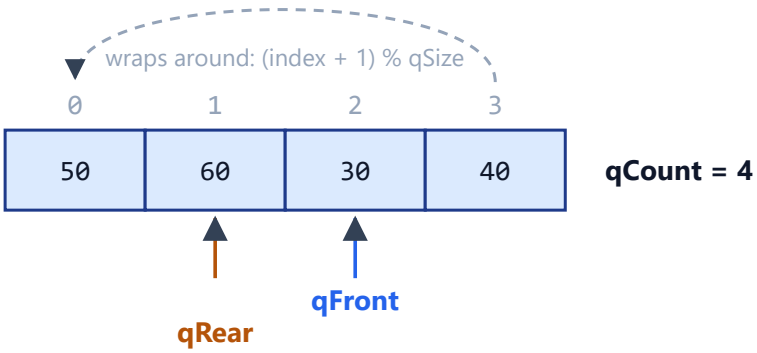
```
for value in [40, 50, 60]:   # the rear pointer goes 3 -> 0 -> 1
    qAdd(value)
qShow()
print()
qAdd(70)                     # the queue is now full
print()
print("Queue in order, front to rear:")
qPrint()
```

OUTPUT

```
index :    0    1    2    3
qData :   50   60   30   40
qFront = 2, qRear = 1, qCount = 4

Queue is full

Queue in order, front to rear:
30
40
50
60
```



The circular queue after the wrap-around. The logical order (30, 40, 50, 60) is not the same as the physical order in the array.

> ⚠️ **Notice:** physically the array holds `50 60 30 40`, but the queue order is `30 40 50 60`, starting from `qFront`. Always read a circular queue **from `qFront`, wrapping round**.

Two things to notice in `qRemove()`: - it **returns** the removed value (or `None` if the queue was empty), so it saves `qData[qFront]` in a variable **before** overwriting it; - unlike `sPop()`, it **does** overwrite the removed slot with `'-'`.

> 📝 **Efficiency:** enqueue and dequeue are both **O(1)** — no items are ever shifted along the array, however long the queue is.

🖍️ Exercise E3 — Peek at the front

Write `qPeek()` that **returns** the item at the front of the queue *without removing it*. If the queue is empty, print `"Queue is empty"` and return `None`.

STARTER CODE

```
def qPeek():
    # Your code here
    pass
```

TESTS (RUN THESE IN THE NOTEBOOK)

```
qInit()
check("qPeek() on an empty queue", qPeek(), None)
qAdd(10); qAdd(20)
check("qPeek() returns the front", qPeek(), 10)
check("qPeek() does not remove", qCount, 2)
qRemove()
check("qPeek() after a remove", qPeek(), 20)
```

🖍️ Exercise E4 — The queue as a list

Write `qToList()` that **returns** a Python list with the items of the queue **in queue order**, from the front to the rear. It must not change the queue.

Hint: copy the loop in `qPrint()`: start at `qFront`, take `qCount` items, and move with `(current + 1) % qSize`.

STARTER CODE

```
def qToList():
    # Your code here
    pass
```

TESTS (RUN THESE IN THE NOTEBOOK)

```
qInit()
check("empty queue", qToList(), [])
qAdd("A"); qAdd("B")
check("two items", qToList(), ["A", "B"])

qInit()
for i in [1, 2, 3, 4]:
    qAdd(i)
qRemove(); qRemove()
qAdd(5); qAdd(6)   # these two wrap around to indexes 0 and 1
check("order after wrap-around", qToList(), [3, 4, 5, 6])
check("the queue is unchanged", qCount, 4)
```

4. The Linked List

A **linked list** is an ordered sequence of **nodes**. Each node stores: - a piece of **data**, and - a **pointer** to the next node.

A **start pointer** tells us where the first node is; the last node's pointer is `-1` (null).

The big idea: the **logical order** of the list (following the pointers) does **not** have to match the **physical order** in memory. So we can insert or delete in the middle **by changing a couple of pointers**, without shifting any data.

	Array	**Linked list**
Access the *i*-th item	direct — O(1)	must follow pointers — O(n)
Insert / delete in the middle	shift items — O(n)	change pointers (after finding the place)
Extra memory	none	one pointer per node

Implementation with two parallel arrays

Variable	**Role**
`lData[i]`	the data stored in node `i`
`lLinks[i]`	the index of the node that comes **after** node `i` (`-1` = end)
`lStart`	index of the **first** node of the list (`-1` = empty list)
`lHeap`	index of the first **free** node — the start of the **free list**

The free list ("heap")

All nodes that are **not** in the list are chained together into a second linked list: the **free list**. At the start *every* node is free, so `lInit()` links them in order: `0 → 1 → 2 → … → 39 → -1`, with `lHeap = 0`.

- `lAllocateNode(value)` **takes** the first node off the free list (like a pop).
- `lReleaseNode(node)` **gives** a deleted node back to the front of the free list (like a push).

> 💡 The free list is itself a **stack of unused nodes**. Every node is always in exactly **one** of the two lists — the data list or the free list. "Heap" here just means "pool of free memory"; it has nothing to do with the heap data structure.

Here is `linked_list.py`, unchanged:

```
# Function to initialize the linked list
def lInit():
    global lSize, lData, lLinks, lStart, lHeap
    lSize = 40  # The maximum size of the linked list
    lData = []  # List to store data values for each node
    lLinks = []  # List to store links (next node indices) for each node
    # Initialize data list with default placeholder values ('-')
    for i in range(lSize):
        lData.append('-')
    # Initialize links list, linking each node to the next one
    for i in range(lSize):
        lLinks.append(i + 1)
    lLinks[lSize - 1] = -1  # The last node points to -1 (end of the heap)
    lStart = -1  # Start of the linked list, -1 indicates the list is empty
    lHeap = 0  # Points to the first free node in the heap

# Function to check if the linked list is empty
def lEmpty():
    return lStart == -1  # Returns True if the list is empty, False otherwise

# Function to check if the linked list is full
def lFull():
    return lHeap == -1  # Returns True if there are no free nodes left

# Function to allocate a new node from the heap
def lAllocateNode(value):
    global lHeap
    newNode = lHeap  # Get the index of the first free node
    lData[lHeap] = value  # Assign the value to the new node
    lHeap = lLinks[lHeap]  # Update the heap pointer to the next free node
    return newNode  # Return the index of the newly allocated node

# Function to release a node back to the heap
def lReleaseNode(node):
    global lHeap
    lData[node] = "-"  # Reset the node's data to the default placeholder
    lLinks[node] = lHeap  # Point the released node to the current heap start
    lHeap = node  # Update the heap pointer to the released node

# Function to search for a value in the linked list
# Returns the index of the node holding the value, or -1 if it is not in the list
def lSearch(value, delete=False):
    global lStart, lHeap
    if lEmpty():
        print("List is empty")  # Message when the list is empty
        return -1
    else:
        prev = -1  # Keeps track of the previous node
        current = lStart  # Start searching from the first node
        nodeFound = False
        # Traverse the list until the value is found or end is reached
        while current != -1 and not nodeFound:
            if lData[current] == value:
                nodeFound = True  # Value found
            else:
                prev = current
                current = lLinks[current]
        if not nodeFound:
            print(value, "not found")  # Value not found in the list
            return -1
```

```
        elif not delete:
            print(value, "found at position", current)  # Value found and no deletion
        else:
            # Handle node deletion
            if prev == -1:  # If the node to delete is the first node
                lStart = lLinks[current]
            else:
                lLinks[prev] = lLinks[current]  # Skip the current node
            lReleaseNode(current)  # Release the deleted node back to the heap
        return current  # Index of the node that was found (or removed)

# Function to remove a node with the specified value
# Returns the index of the removed node, or -1 if the value was not found
def lRemove(value):
    return lSearch(value, True)  # Search and delete the node with the given value

# Function to insert a value into the linked list
def lInsert(value, mode=0):
    '''
    Insert a value into the linked list based on the mode:
    0 - Insert at the start.
    1 - Insert in ascending order.
    2 - Insert in descending order.
    '''
    global lStart, lHeap
    if lFull():
        print("List is full")  # Message when the list is full
    else:
        newNode = lAllocateNode(value)  # Allocate a new node
        prev = -1  # Keeps track of the previous node
        current = lStart  # Start searching from the first node
        nodeFound = False
        # Traverse the list to find the appropriate position for insertion
        # (mode 0 always inserts at the start, so it skips the traversal)
        while mode != 0 and current != -1 and not nodeFound:
            if (mode == 1 and lData[current] > value) or (mode == 2 and lData[current] < value):
                nodeFound = True  # Found the position for insertion
            else:
                prev = current
                current = lLinks[current]
        if mode == 0 or prev == -1:  # Insert at the start or as the first node
            lLinks[newNode] = lStart
            lStart = newNode
        else:  # Insert at the appropriate position
            lLinks[newNode] = current
            lLinks[prev] = newNode

# Function to print the linked list
def lPrint():
    global lStart
    if lEmpty():
        print("List is empty")  # Message when the list is empty
    else:
        current = lStart  # Start from the first node
        while current != -1:
            nextNode = lLinks[current]  # Get the next node index
            if nextNode != -1:
                print(lData[current], "->", lData[nextNode])  # Print current -> next
            else:
```

```
                print(lData[current], "-> -1")  # End of the list
            current = nextNode  # Move to the next node
```

Two helpers: one to show the arrays, one to return the values in logical order (useful for testing):

```
def lShow(n=6):
    """Show the first n nodes of the two arrays, plus both start pointers."""
    print("index | lData | lLinks")
    print("------+-------+-------")
    for i in range(n):
        print(f"{i:>5} | {str(lData[i]):>5} | {lLinks[i]:>6}")
    print(f"lStart = {lStart}, lHeap = {lHeap}")

def lToList():
    """Return the values in logical (linked) order as a Python list."""
    values = []
    current = lStart
    while current != -1:
        values.append(lData[current])
        current = lLinks[current]
    return values
```

The initial state: every node is free

```
lInit()
lShow()
```

OUTPUT

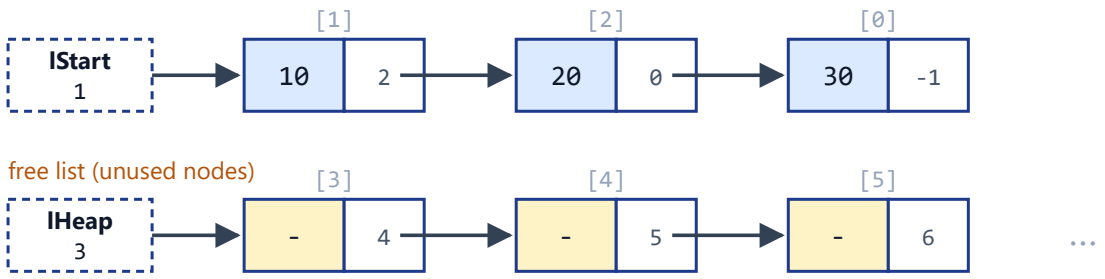
```
index | lData | lLinks
------+-------+-------
    0 |     - |      1
    1 |     - |      2
    2 |     - |      3
    3 |     - |      4
    4 |     - |      5
    5 |     - |      6
lStart = -1, lHeap = 0
```

Inserting in ascending order

`lInsert(value, mode)` supports three modes: `0` = insert at the start, `1` = keep the list in **ascending** order, `2` = keep it in **descending** order. Let's insert 30, 10 and 20 in ascending order:

```
lInit()
for value in [30, 10, 20]:
    lInsert(value, 1)
lShow()
print("Logical order:", lToList())
```

```
OUTPUT
index | lData | lLinks
------+-------+-------
    0 |    30 |     -1
    1 |    10 |      2
    2 |    20 |      0
    3 |     - |      4
    4 |     - |      5
    5 |     - |      6
lStart = 1, lHeap = 3
Logical order: [10, 20, 30]
```



Logical view: lStart → [1] 10 → [2] 20 → [0] 30. The free list starts at node 3.

Physically the data is stored as `30, 10, 20` (in the order it arrived), but following the pointers from `lStart` gives `10 → 20 → 30`.

How `lInsert` finds the right place — the *prev / current* pattern

`lInsert` walks along the list with **two** pointers: - `current` — the node we are looking at, - `prev` — the node just before it.

It stops when it finds the first node whose value is bigger than the new one (mode 1). The new node then goes **between** `prev` **and** `current`:

```
lLinks[newNode] = current     # 1) the new node points to current
lLinks[prev]    = newNode     # 2) prev now points to the new node
```

If `prev` is still `-1`, the new node goes at the very start, so `lStart` changes instead.

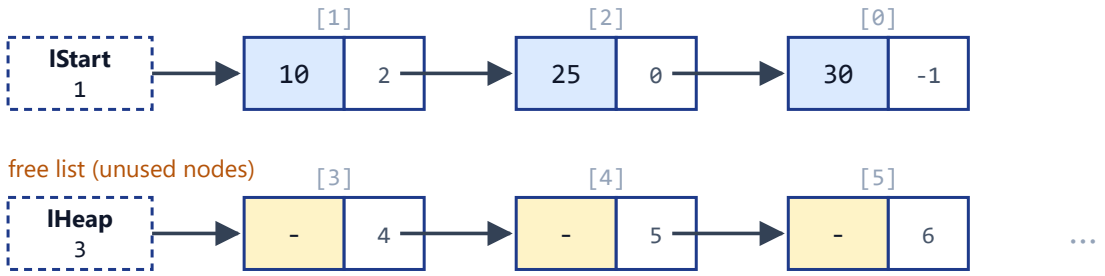
> ⚠️ **Exam tip:** the order of those two assignments matters. If you overwrite `lLinks[prev]` first, you lose the only reference to `current`!

Deleting, and recycling the node

```
lRemove(20)          # node 2 is unlinked and returned to the free list
lShow()
print("Logical order:", lToList())
print()
lInsert(25, 1)       # the new value reuses node 2
lShow()
print("Logical order:", lToList())
```

```
OUTPUT
index | lData | lLinks
------+-------+-------
    0 |    30 |     -1
    1 |    10 |      0
    2 |     - |      3
    3 |     - |      4
    4 |     - |      5
    5 |     - |      6
lStart = 1, lHeap = 2
Logical order: [10, 30]

index | lData | lLinks
------+-------+-------
    0 |    30 |     -1
    1 |    10 |      2
    2 |    25 |      0
    3 |     - |      4
    4 |     - |      5
    5 |     - |      6
lStart = 1, lHeap = 3
Logical order: [10, 25, 30]
```



After deleting 20 and inserting 25: node 2 was released to the free list and then allocated again.

The three insert modes and searching

```
for mode in [0, 1, 2]:
    lInit()
    for value in [3, 1, 2]:
        lInsert(value, mode)
    print("mode", mode, "->", lToList())

print()
position = lSearch(3)
print("lSearch returned", position)
position = lSearch(99)
print("lSearch returned", position)
```

OUTPUT

```
mode 0 -> [2, 1, 3]
mode 1 -> [1, 2, 3]
mode 2 -> [3, 2, 1]

3 found at position 0
lSearch returned 0
99 not found
lSearch returned -1
```

`lSearch` prints a message **and returns** the index of the node, or `-1` if the value isn't there — just like a search function in the exam. `lRemove` returns the index of the node it removed (or `-1`).

> 📝 **Efficiency:** searching, ordered insertion (modes 1 and 2) and deletion are **O(n)** because we may have to follow every pointer. Inserting at the start (mode 0) is **O(1)**: notice the `mode != 0` test in the `while` loop of `lInsert`, which skips the traversal completely.

🖍️ Exercise E5 — Count the nodes

Write `lCount()` that returns how many nodes are in the list, by following the pointers from `lStart` until you reach `-1`.

STARTER CODE

```
def lCount():
    # Your code here
    pass
```

TESTS (RUN THESE IN THE NOTEBOOK)

```
lInit()
check("empty list", lCount(), 0)
for v in [5, 3, 8, 1, 9]:
    lInsert(v, 1)
check("five nodes", lCount(), 5)
lRemove(8)
check("after one removal", lCount(), 4)
```

🖍️ Exercise E6 — Reverse the list in place

Write `lReverse()` that reverses the list **by changing the pointers only** — no data moves and no new nodes are allocated.

Hint: walk along the list with three variables, `prev`, `current` and `nextNode`. For each node, save its next node, point it back to `prev`, then move all three along. At the end `lStart` must point to the old last node.

STARTER CODE

```
def lReverse():
    # Your code here
    pass
```

TESTS (RUN THESE IN THE NOTEBOOK)

```
lInit()
for v in [1, 2, 3, 4]:
    lInsert(v, 1)
lReverse()
check("reverse 1-2-3-4", lToList(), [4, 3, 2, 1])
check("data did not move", lData[:4], [1, 2, 3, 4])

lInit()
lReverse()
check("reverse an empty list", lToList(), [])
```

🖍️ Exercise E7 — Insert at the end

Write `lAppend(value)` that adds `value` as the **last** node of the list. If the list is full, print `"List is full"` and do nothing.

Hint: get a node with `lAllocateNode(value)` and set its link to `-1`. If the list is empty, the new node becomes `lStart`. Otherwise follow the pointers until you reach the node whose link is `-1`, and make it point to the new node.

STARTER CODE

```
def lAppend(value):
    # Your code here
    pass
```

TESTS (RUN THESE IN THE NOTEBOOK)

```
lInit()
for v in [3, 1, 2]:
    lAppend(v)
check("append 3, 1, 2", lToList(), [3, 1, 2])
lInsert(0, 0)
lAppend(9)
check("mixed with lInsert", lToList(), [0, 3, 1, 2, 9])

lInit()
for i in range(lSize + 1):   # one too many
    lAppend(i)
check("full list keeps lSize nodes", len(lToList()), lSize)
```

5. The Binary Search Tree

A **binary tree** is made of **nodes**; each node has data and **at most two children**, a *left* one and a *right* one. The top node is the **root**; nodes with no children are **leaves**.

A **binary search tree (BST)** adds an ordering rule:

> for every node, values in its **left** subtree are **smaller**, and values in its **right** subtree are **greater or equal**.

This rule means that at each node we can throw away half of the remaining tree — like a binary search.

Where trees are used: fast searching and sorting, database indexes, file-system hierarchies, expression trees, decision trees.

Implementation with three parallel arrays

Variable	Role
`bData[i]`	the data in node `i`
`bLeft[i]`	index of the **left** child of node `i` (`-1` = none)
`bRight[i]`	index of the **right** child of node `i` (`-1` = none)
`bRoot`	index of the root (`-1` = empty tree)
`bHeap`	index of the first **free** node

> 💡 **Clever trick:** while a node is free, it doesn't need children, so the code **reuses `bLeft` as the "next free node" pointer**. That's why `bInit()` sets `bLeft[i] = i + 1`, and why `bAllocateNode` resets both child pointers to `-1` as soon as it hands a node out.

Here is `binary_tree.py`, unchanged:

```
# Function to initialize the binary tree
def bInit():
    global bSize, bData, bLeft, bRight, bRoot, bHeap
    bSize = 10  # Maximum number of nodes in the tree
    bRoot = -1  # Index of the root node, -1 indicates the tree is empty
    bHeap = 0  # Index of the first available free node in the heap
    bData = []  # List to store data values for each node
    bRight = []  # List to store the right child index for each node
    bLeft = []  # List to store the left child index for each node

    # Initialize data list with default placeholder values ('-')
    for i in range(bSize):
        bData.append('-')
    # Initialize left and right child lists
    for i in range(bSize):
        bRight.append(-1)  # Initially, no right child for any node
        bLeft.append(i + 1)  # Each node points to the next as a free node
    bLeft[bSize - 1] = -1  # The last node in the heap points to -1 (end of heap)

# Function to check if the binary tree is empty
def bEmpty():
    return bRoot == -1  # Returns True if the tree has no root node

# Function to check if the binary tree is full
def bFull():
    return bHeap == -1  # Returns True if there are no free nodes left in the heap

# Function to allocate a new node from the heap
def bAllocateNode(value):
    global bHeap, bRight, bLeft
    newNode = bHeap  # Get the index of the first free node
    bData[bHeap] = value  # Assign the value to the new node
    bHeap = bLeft[bHeap]  # Update the heap pointer to the next free node
    bLeft[newNode] = -1  # Initialize the left child of the new node as -1
    bRight[newNode] = -1  # Initialize the right child of the new node as -1
    return newNode  # Return the index of the newly allocated node

# Function to search for a value in the binary tree
# Returns the index of the node holding the value, or -1 if it is not in the tree
def bSearch(value):
    global bRoot, bHeap
    if bEmpty():
        print("Tree is empty")  # Message when the tree is empty
        return -1
    else:
        current = bRoot  # Start from the root node
        nodeFound = False
        # Traverse the tree to find the value
        while current != -1 and not nodeFound:
            if bData[current] == value:  # Value found
                nodeFound = True
            elif value < bData[current]:  # Move to the left child
                current = bLeft[current]
            else:  # Move to the right child
                current = bRight[current]
        if not nodeFound:
            print(value, "not found")  # Value not found in the tree
            return -1
        else:
            print(value, "found at position", current)  # Value found
```

```
            return current

# Function to insert a value into the binary tree
def bInsert(value):
    global bRoot, bHeap
    if bFull():
        print("Tree is full")  # Message when the tree is full
    else:
        newNode = bAllocateNode(value)  # Allocate a new node
        if bEmpty():
            bRoot = newNode  # If tree is empty, make the new node the root
        else:
            current = bRoot  # Start from the root node
            parent = -1  # Keeps track of the parent node
            # Traverse the tree to find the correct position for insertion
            while current != -1:
                parent = current
                if value < bData[current]:  # Move to the left child
                    current = bLeft[current]
                else:  # Move to the right child
                    current = bRight[current]
            # Insert the new node as the left or right child of the parent
            if value < bData[parent]:
                bLeft[parent] = newNode
            else:
                bRight[parent] = newNode
```

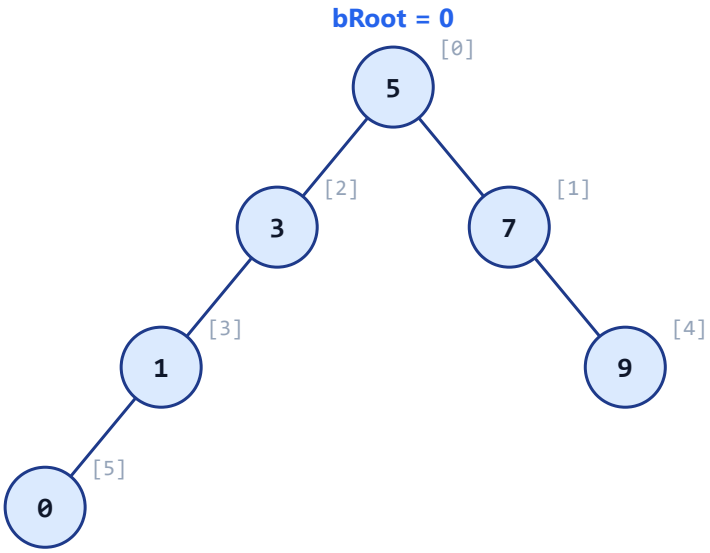
```
def bShow(n=8):
    """Show the first n nodes of the three arrays, plus the root and free pointers."""
    print("index | bData | bLeft | bRight")
    print("------+-------+-------+-------")
    for i in range(n):
        print(f"{i:>5} | {str(bData[i]):>5} | {bLeft[i]:>5} | {bRight[i]:>6}")
    print(f"bRoot = {bRoot}, bHeap = {bHeap}")
```

Building a tree

```
bInit()
for value in [5, 7, 3, 1, 9, 0]:
    bInsert(value)
bShow()
```

OUTPUT

```
index | bData | bLeft | bRight
------+-------+-------+-------
    0 |     5 |     2 |      1
    1 |     7 |    -1 |      4
    2 |     3 |     3 |     -1
    3 |     1 |     5 |     -1
    4 |     9 |    -1 |     -1
    5 |     0 |    -1 |     -1
    6 |     - |     7 |     -1
    7 |     - |     8 |     -1
bRoot = 0, bHeap = 6
```



The tree built by inserting 5, 7, 3, 1, 9, 0. The small grey numbers are the array indexes of the nodes.

Read the table together with the picture: node `[0]` holds 5, its left child is node `2` (value 3) and its right child is node `1` (value 7). Nodes 6 and 7 are still free — their `bLeft` values form the free list `6 → 7 → 8 → …`.

How insertion and search work

Both start at the root and repeat: *if the value is smaller go left, otherwise go right* **— until they reach a `-1`. -** search **stops early if it finds the value; -** insert** remembers the last real node (`parent`) and hangs the new node on its left or right.

```
bSearch(1)     # path: 5 -> 3 -> 1
bSearch(9)     # path: 5 -> 7 -> 9
position = bSearch(-1)    # path: 5 -> 3 -> 1 -> 0 -> (no left child) -> not found
print("bSearch returned", position)
```

OUTPUT

```
1 found at position 3
9 found at position 4
-1 not found
bSearch returned -1
```

Like `lSearch`, `bSearch` prints a message **and returns** the index of the node, or `-1` if the value is not in the tree.

Traversals

A **traversal** visits every node once. For binary trees there are three classic recursive orders:

Traversal	Order	Use
Pre-order	node → left → right	copying a tree, prefix (Polish) notation
In-order	left → node → right	outputs a BST in **sorted order**
Post-order	left → right → node	deleting a tree, postfix (Reverse Polish) notation

Here is pre-order as a worked example. Notice the **base case** (`node == -1`, an empty subtree) and the two **recursive calls**:

```
def bPreOrder(node):
    """Visit the node, then its left subtree, then its right subtree."""
    if node != -1:
        print(bData[node], end=" ")
        bPreOrder(bLeft[node])
        bPreOrder(bRight[node])

bPreOrder(bRoot)
```

```
OUTPUT

5 3 1 0 7 9
```

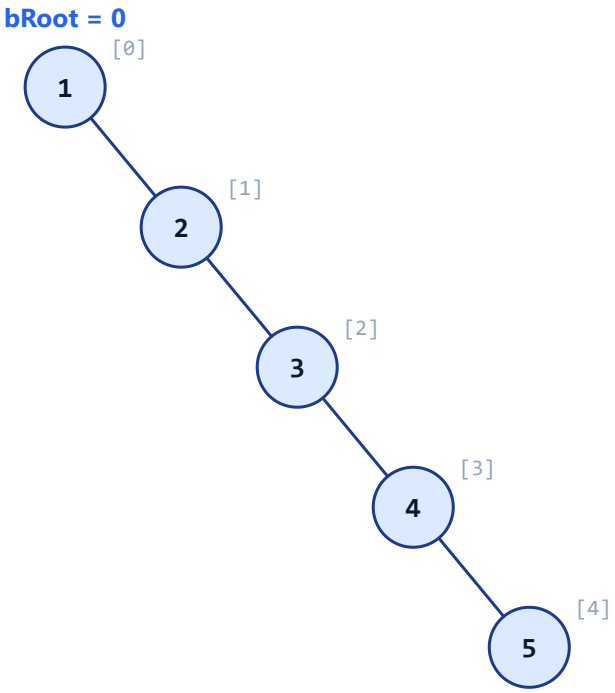
When a BST goes wrong

Searching a BST is **O(log n)** when the tree is **balanced**. But the shape depends on the **order of insertion**. Watch what happens if the data arrives already sorted:

```
bInit()
for value in [1, 2, 3, 4, 5]:
    bInsert(value)
bShow(6)
```

```
OUTPUT

index | bData | bLeft | bRight
------+-------+-------+-------
    0 |     1 |    -1 |      1
    1 |     2 |    -1 |      2
    2 |     3 |    -1 |      3
    3 |     4 |    -1 |      4
    4 |     5 |    -1 |     -1
    5 |     - |     6 |     -1
bRoot = 0, bHeap = 5
```



Inserting sorted data gives a degenerate tree: every node only has a right child.

The tree has become a **linked list** in disguise, and searching is back to **O(n)**. (Self-balancing trees such as AVL trees fix this — beyond the A Level spec, but worth knowing.)

> 📝 Insert and search in a BST are **O(log n)** on average and **O(n)** in the worst case.

🖍️ Exercise E8 — Smallest and largest values

Write `bMin()` and `bMax()` that **return** the smallest and largest value in the tree, or `None` if the tree is empty.

Hint: where is the smallest value in a BST? Keep going left from the root until there is no left child.

> 🤔 **Careful with names:** the node you reach is the **leftmost node**, which is *not always a leaf*. Insert 10, 5, 7: the minimum is 5, but node 5 still has a right child (7)!

STARTER CODE

```
def bMin():
    # Your code here
    pass

def bMax():
    # Your code here
    pass
```

TESTS (RUN THESE IN THE NOTEBOOK)

```
bInit()
check("bMin() on an empty tree", bMin(), None)
for v in [50, 30, 70, 20, 40, 60, 80]:
    bInsert(v)
check("bMin()", bMin(), 20)
check("bMax()", bMax(), 80)

bInit()
for v in [10, 5, 7]:
    bInsert(v)
check("minimum that is not a leaf", bMin(), 5)
```

🖍️ Exercise E9 — In-order and post-order traversals

Using `bPreOrder` as a model, write two **recursive** functions that **return a Python list** of the values (instead of printing them):

- `bInOrder(node)` — left, node, right
- `bPostOrder(node)` — left, right, node

Call them with `bRoot`. For a BST, the in-order list should come out **sorted**.

STARTER CODE

```
def bInOrder(node):
    # Your code here
    pass

def bPostOrder(node):
    # Your code here
    pass
```

TESTS (RUN THESE IN THE NOTEBOOK)

```
bInit()
for v in [5, 7, 3, 1, 9, 0]:
    bInsert(v)
check("in-order is sorted", bInOrder(bRoot), [0, 1, 3, 5, 7, 9])
check("post-order", bPostOrder(bRoot), [0, 1, 3, 9, 7, 5])
bInit()
check("empty tree", bInOrder(bRoot), [])
```

🖍️ Exercise E10 — ⭐ Challenge: height of the tree

The **height** of a tree is the number of nodes on the longest path from the root down to a leaf (an empty tree has height 0). Write a recursive `bHeight(node)`.

Hint: the height of a node is 1 + the larger of the heights of its two subtrees.

STARTER CODE

```
def bHeight(node):
    # Your code here
    pass
```

TESTS (RUN THESE IN THE NOTEBOOK)

```
bInit()
check("empty tree", bHeight(bRoot), 0)
for v in [5, 7, 3, 1, 9, 0]:
    bInsert(v)
check("tree 5,7,3,1,9,0", bHeight(bRoot), 4)
bInit()
for v in [1, 2, 3, 4, 5]:
    bInsert(v)
check("degenerate tree", bHeight(bRoot), 5)
```

6. Exam-style tracing questions

In the exam you won't have a computer: you will be asked to **trace** the arrays and pointers by hand. Do these **on paper first**, then check your answers.

🖋️ T1 — Stack

`sInit()` is called, followed by:

`sPush("A"), sPush("B"), sPush("C"), sPop(), sPush("D"), sPop(), sPop()`

(a) Which values are returned by the three pops, in order? **(b)** What is the final value of `sTop`? **(c)** What do `sData[0]` to `sData[3]` contain at the end?

🖋️ T2 — Circular queue

`qInit()` is called (so `qSize = 4`), followed by:

`qAdd("A"), qAdd("B"), qAdd("C"), qRemove(), qRemove(), qAdd("D"), qAdd("E"), qAdd("F")`

(a) Give the final values of `qFront`, `qRear` and `qCount`. **(b)** Give the contents of `qData[0]` to `qData[3]`. **(c)** What happens if `qAdd("G")` is now called?

🖋️ T3 — Linked list

`lInit()` is called, followed by:

`lInsert(40, 1), lInsert(20, 1), lInsert(30, 1), lRemove(20), lInsert(10, 1)`

Copy and complete the table for indexes 0–4, and give the final values of `lStart` and `lHeap`.

index	lData	lLinks
0		
1		
2		
3		
4		

🖋️ T4 — Binary search tree

`bInit()` is called, then these values are inserted in this order:

`50, 30, 70, 20, 40, 60, 80, 35`

(a) Draw the tree. **(b)** Complete `bData`, `bLeft` and `bRight` for indexes 0–9, and give `bRoot` and `bHeap`. **(c)** List the nodes visited when searching for 35. **(d)** Give the in-order and pre-order traversals.

7. Summary

ADT	Order	Pointers	Add	Remove	Search
Stack	LIFO	`sTop`	O(1) push	O(1) pop	—
Circular queue	FIFO	`qFront`, `qRear` (+ `qCount`)	O(1) at rear	O(1) at front	—
Linked list	by links (e.g. sorted)	`lStart`, `lHeap`, `lLinks[]`	O(n) sorted, O(1) at start	O(n)	O(n) linear
Binary search tree	left < node ≤ right	`bRoot`, `bHeap`, `bLeft[]`, `bRight[]`	O(log n) avg	—	O(log n) avg, O(n) worst

Checklist before the exam

- [ ] I can explain the difference between an ADT and its implementation.
- [ ] I know what `-1` means for every pointer, and the initial value of every pointer.
- [ ] I can trace a stack and show that popping doesn't erase data.
- [ ] I can use `% qSize` to wrap a circular queue, and explain why `qCount` is useful.
- [ ] I can explain the free list: how nodes are allocated and released.
- [ ] I can insert into and delete from a linked list with the *prev / current* pattern.
- [ ] I can draw a BST from an insertion order **and** fill in its `bLeft` / `bRight` arrays.
- [ ] I can write pre-, in- and post-order traversals recursively.
- [ ] I can say whether an operation is O(1), O(n) or O(log n) by looking at its loops.

Common mistakes

1. Forgetting `global` when a function changes `sTop`, `qFront`, `lStart`…
2. Not checking `Empty()` / `Full()` before removing / adding (underflow / overflow).
3. Reading a circular queue from index 0 instead of from `qFront`.
4. Updating linked-list pointers in the wrong order and losing part of the list.
5. Forgetting the special case where the **first** node changes (`prev == -1`).
6. Thinking that a deleted node disappears — in fact it goes back to the **free list**.

Appendix A. Solutions to the exercises

Each solution is followed by the output of the test cells, all passing.

E1 — Reverse a string with a stack

```
def reverseString(string):
    sInit()                      # start with an empty stack
    for char in string:
        sPush(char)              # push every character
    reversedString = ""
    while not sEmpty():
        reversedString += sPop() # the last character in comes out first
    return reversedString
```

```
OUTPUT
✅ reverseString('hello')
✅ reverseString('')
✅ reverseString('Y13')
```

E2 — Balanced brackets

```
def isBalanced(expression):
    pairs = {")": "(", "]": "[", "}": "{"}   # closing bracket -> its opener
    sInit()
    for char in expression:
        if char in "([{":
            sPush(char)
        elif char in pairs:
            # nothing to match, or the wrong kind of bracket
            if sEmpty() or sPop() != pairs[char]:
                return False
    return sEmpty()   # any brackets left open?
```

```
OUTPUT
✅ (())
✅ (()
✅ )(
✅ {[()]}
✅ ([)]
✅ a*(b+c[i])
```

E3 — Peek at the front

```
def qPeek():
    if qEmpty():
        print("Queue is empty")
        return None
    return qData[qFront]   # front item; no pointers change
```

OUTPUT

```
Queue is empty
✅ qPeek() on an empty queue
✅ qPeek() returns the front
✅ qPeek() does not remove
✅ qPeek() after a remove
```

E4 — The queue as a list

```
def qToList():
    items = []
    current = qFront
    for i in range(qCount):
        items.append(qData[current])
        current = (current + 1) % qSize   # move to the next slot circularly
    return items
```

OUTPUT

```
✅ empty queue
✅ two items
✅ order after wrap-around
✅ the queue is unchanged
```

E5 — Count the nodes

```
def lCount():
    count = 0
    current = lStart
    while current != -1:            # follow the pointers to the end
        count += 1
        current = lLinks[current]
    return count
```

OUTPUT

```
✅ empty list
✅ five nodes
✅ after one removal
```

E6 — Reverse the list in place

```
def lReverse():
    global lStart
    prev = -1
    current = lStart
    while current != -1:
        nextNode = lLinks[current]   # remember where to go next
        lLinks[current] = prev       # turn this node's arrow around
        prev = current
        current = nextNode
    lStart = prev                    # the old last node is the new first node
```

OUTPUT

```
✅ reverse 1-2-3-4
✅ data did not move
✅ reverse an empty list
```

E7 — Insert at the end

```
def lAppend(value):
    global lStart
    if lFull():
        print("List is full")
        return
    newNode = lAllocateNode(value)
    lLinks[newNode] = -1             # it will be the last node
    if lEmpty():
        lStart = newNode
    else:
        current = lStart
        while lLinks[current] != -1:  # stop at the last node
            current = lLinks[current]
        lLinks[current] = newNode
```

OUTPUT

```
✅ append 3, 1, 2
✅ mixed with lInsert
List is full
✅ full list keeps lSize nodes
```

E8 — Smallest and largest values

```
def bMin():
    if bEmpty():
        return None
    current = bRoot
    while bLeft[current] != -1:      # keep going left
        current = bLeft[current]
    return bData[current]

def bMax():
    if bEmpty():
        return None
    current = bRoot
    while bRight[current] != -1:     # keep going right
        current = bRight[current]
    return bData[current]
```

OUTPUT

```
✅ bMin() on an empty tree
✅ bMin()
✅ bMax()
✅ minimum that is not a leaf
```

E9 — In-order and post-order traversals

```
def bInOrder(node):
    if node == -1:                  # base case: empty subtree
        return []
    return bInOrder(bLeft[node]) + [bData[node]] + bInOrder(bRight[node])

def bPostOrder(node):
    if node == -1:
        return []
    return bPostOrder(bLeft[node]) + bPostOrder(bRight[node]) + [bData[node]]
```

OUTPUT

```
✅ in-order is sorted
✅ post-order
✅ empty tree
```

E10 — ⭐ Challenge: height of the tree

```
def bHeight(node):
    if node == -1:
        return 0
    return 1 + max(bHeight(bLeft[node]), bHeight(bRight[node]))
```

OUTPUT

```
✅ empty tree
✅ tree 5,7,3,1,9,0
✅ degenerate tree
```

Appendix B. Answers to the tracing questions

T1 — Stack

(a) `C`, `D`, `B` **(b)** `sTop = 0` **(c)** `A B D -`

`D` was pushed into index 2, **overwriting** `C`. After the last two pops, `B` and `D` are still physically in the array but are above `sTop`.

CHECK

```
sInit()
sPush("A"); sPush("B"); sPush("C")
print(sPop())
sPush("D")
print(sPop())
print(sPop())
sShow(4)
```

OUTPUT

```
C
D
B
index | sData
------+------
    3 | -
    2 | D
    1 | B
    0 | A      <-- sTop
sTop = 0
```

T2 — Circular queue

(a) `qFront = 2`, `qRear = 1`, `qCount = 4` **(b)** `E F C D` (queue order: `C D E F`) **(c)** the queue is full, so `Queue is full` is printed and nothing changes.

CHECK

```
qInit()
qAdd("A"); qAdd("B"); qAdd("C")
qRemove(); qRemove()
qAdd("D"); qAdd("E"); qAdd("F")
qShow()
qAdd("G")
```

OUTPUT

```
index :    0    1    2    3
qData :    E    F    C    D
qFront = 2, qRear = 1, qCount = 4
Queue is full
```

T3 — Linked list

index	lData	lLinks
0	40	-1
1	10	2
2	30	0
3	-	4
4	-	5

`lStart = 1`, `lHeap = 3`. Logical order: `10 → 30 → 40`. Node 1 was freed when 20 was removed and then **reused** for 10.

CHECK

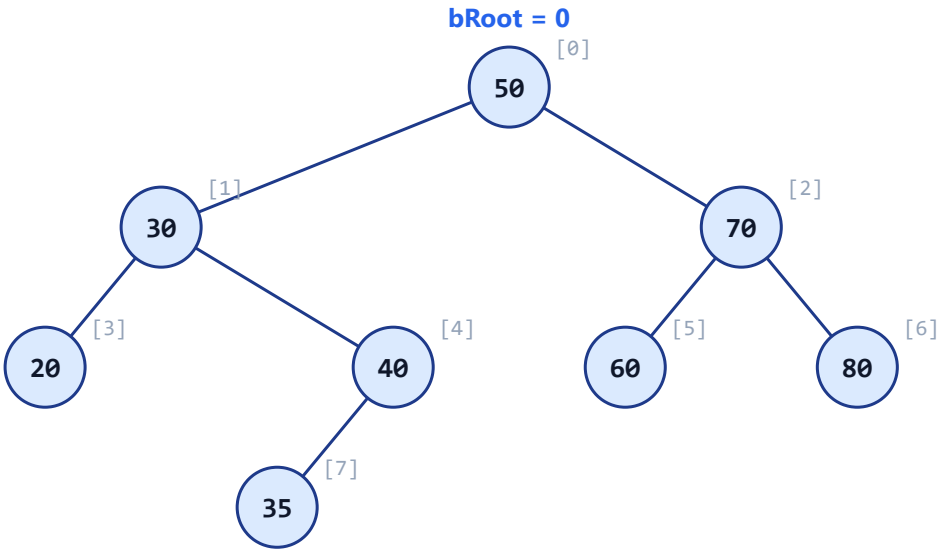
```
lInit()
lInsert(40, 1); lInsert(20, 1); lInsert(30, 1)
lRemove(20)
lInsert(10, 1)
lShow(5)
print("Logical order:", lToList())
```

OUTPUT

```
index | lData | lLinks
------+-------+-------
    0 |    40 |     -1
    1 |    10 |      2
    2 |    30 |      0
    3 |     - |      4
    4 |     - |      5
lStart = 1, lHeap = 3
Logical order: [10, 30, 40]
```

T4 — Binary search tree

(b) see the table below. `bRoot = 0`, `bHeap = 8`; nodes 8 and 9 are free (`bLeft[8] = 9`, `bLeft[9] = -1`). **(c)** 50 → 30 → 40 → 35 **(d)** in-order: `20 30 35 40 50 60 70 80`; pre-order: `50 30 20 40 35 70 60 80`



The tree for T4.

CHECK

```
bInit()
for v in [50, 30, 70, 20, 40, 60, 80, 35]:
    bInsert(v)
bShow(10)
print("In-order: ", bInOrder(bRoot))
print("Pre-order: ", end="")
bPreOrder(bRoot)
```

OUTPUT

```
index | bData | bLeft | bRight
------+-------+-------+-------
    0 |    50 |     1 |      2
    1 |    30 |     3 |      4
    2 |    70 |     5 |      6
    3 |    20 |    -1 |     -1
    4 |    40 |     7 |     -1
    5 |    60 |    -1 |     -1
    6 |    80 |    -1 |     -1
    7 |    35 |    -1 |     -1
    8 |     - |     9 |     -1
    9 |     - |    -1 |     -1
bRoot = 0, bHeap = 8
In-order:  [20, 30, 35, 40, 50, 60, 70, 80]
Pre-order: 50 30 20 40 35 70 60 80
```